Numerical Methods for Partial Differential Equations

Gabriele Basciu

Project 1 – Diffusion equation | Project 2 – Convection-diffusion | Project 3 – Parabolic equation

P1 Galerkin Finite Element Method

Department of Mathematics “Tullio Levi-Civita”
University of Padua

July 17, 2026

Contents

1 Introduction 3

2 Project 1 – Diffusion Equation (Poisson) 4

2.1 Problem formulation 4

2.2 Galerkin finite element method 4

2.3 Architecture of the implemented code 5

2.4 Solving the linear system: backslash and PCG 5

2.5 Numerical results and convergence analysis 6

3 Project 2 – Convection-Diffusion Equation 10

3.1 Problem formulation 10

3.2 Galerkin formulation and transport matrix 10

3.3 Implementation notes 10

3.4 Solution without stabilization: onset of oscillations 11

3.5 Streamline-Upwind Diffusion (SUD) stabilization 12

4 Project 3 – Parabolic Equation 14

4.1 Problem formulation 14

4.2 FEM discretization and θ-method 14

4.3 Implementation notes: solveParabolic.m 14

4.4 Spatial convergence 14

4.5 Direct temporal convergence 15

4.6 Self-convergence (Richardson) in time 16

4.7 Stability of explicit Euler and the parabolic CFL limit 17

5 Conclusions 19

1 Introduction

This report brings together the three laboratory projects carried out as part of the Numerical Methods for Partial Differential Equations course, devoted to the approximation, via the P1 Galerkin Finite Element Method (FEM), of three increasingly complex classes of partial differential problems: a purely elliptic diffusion problem (Poisson equation, Project 1), an elliptic convection-diffusion problem (Project 2) and a time-dependent parabolic diffusion problem (Project 3).

The three codes share the same basic infrastructure: reading the triangulation and boundary data from file, computing the geometric coefficients of the linear, locally supported basis functions on each triangle, element-wise assembly of the global matrices (stiffness, mass, transport, stabilization) and imposition of the Dirichlet conditions via the penalty method. Onto this common base are grafted, in turn, the specific elements of each project: in Project 1 the comparison between the direct solution and the preconditioned conjugate gradient method (PCG); in Project 2 the non-symmetric transport matrix and the Streamline-Upwind Diffusion (SUD) stabilization; in Project 3 the time discretization with the θ-method and the study of stability and of the spatial and temporal convergence orders.

For each project the following are reported: the formulation of the continuous problem, the weak formulation and its Galerkin discretization, a description of the implementation choices made in MATLAB (with direct reference to the functions written), and finally the numerical results obtained on the provided sequence of meshes (mesh0–mesh4, with characteristic parameter h halved at each refinement), accompanied by the plots produced by the code and a critical comment on the consistency between theoretical and observed convergence orders.

2 Project 1 – Diffusion Equation (Poisson)

2.1 Problem formulation

The first project concerns the numerical solution of the Poisson equation with mixed Dirichlet and Neumann conditions:

$$-div(\nabla u) = f \ in \ \Omega = (-1,1)\times(-1,1)$$

with unit scalar diffusion coefficient. The boundary conditions are assigned as follows: homogeneous Dirichlet u = 0 on the horizontal sides y = ±1 (ΓD), and Neumann on the vertical sides x = ±1 (ΓN), with fluxes

$$q1(y) = -\pi \cos(\pi y/2) \ \ on \ x = 1, \quad q2(y) = \pi \cos(\pi y/2) \ \ on \ x = -1.$$

The forcing term and the exact solution, used to compute the error, are related by f = −Δu with

$$u(x,y) = \sin(\pi x)\cos(\pi y/2), \quad f(x,y) = (5/4)\pi^2 \sin(\pi x)\cos(\pi y/2).$$

The problem is solved on the sequence of five triangulations mesh0,…,mesh4, obtained by successive refinements with characteristic parameter h halved at each level.

2.2 Galerkin finite element method

Having introduced the finite element space Vh = span{ϕ1,…,ϕn} ⊂ $H^1(\Omega)$ generated by P1 basis functions (piecewise linear, with local support on a single triangle), the approximate solution uh = Σ ujϕj satisfies the Galerkin weak formulation, which leads to the symmetric, positive-definite linear system A u = b, with stiffness matrix A and right-hand-side vector b defined by the element-wise assembly of the local contributions.

2.2.1 Local basis functions and geometric coefficients

For every triangle T with nodes i, j, k the basis function ϕi(x,y) = (ai + bix + ciy)/(2Δ), where Δ is the area of the triangle and (a,b,c) are determined by imposing the interpolation property at the vertices. This calculation is carried out by the function localBasis.m, which for each element computes the area (via the determinant of the homogeneous-coordinate matrix) and the angular coefficients bi = yj − yk, ci = xk − xj (and cyclic permutations), which are then reused by all the assembly routines.

2.2.2 Assembly of the stiffness matrix

$$a_{ij} = \Sigma^T (1/4\Delta^T)(b_i{}^T b_j{}^T + c_i{}^T c_j{}^T)$$

The function stiffBuild.m builds the sparse Nodes×Nodes matrix by computing, for each element, the local 3×3 matrix according to the formula above and adding it (element-wise assembly) to the global positions indicated by the connectivity triang(el,:). The resulting matrix is sparse, symmetric and (after the boundary conditions are imposed) positive definite.

2.2.3 Assembly of the right-hand-side vector

The vector b receives two contributions. The first, relating to the forcing term, is computed in rhsBuild.m by applying the trapezoidal rule, which for P1 elements amounts to weighting the nodal value of the forcing term by the nodal area Ai (one third of the area of each triangle incident on the node, computed by calculateNodalArea.m):

$$bi = Ai \cdot f(xi, yi).$$

The second contribution, relating to the Neumann flux, is added only at the ΓN nodes and is given by bi += q(yi)·li, where li is the average length of the two boundary sides incident on node i, computed by calculateBoundaryLength.m by traversing all the sides of the triangulation and isolating those with both endpoints on x = 1 or x = −1. The sign of the flux (q1 or q2) is assigned in rhsBuild.m based on the x coordinate of the node.

2.2.4 Imposition of Dirichlet conditions

The Dirichlet conditions are imposed using the penalty method: the function imposeBC.m adds the value 1/ε (with $\varepsilon = 10^{-15}$) to the diagonal entry of the Dirichlet node and the term DirVal(i)/ε to the corresponding component of the right-hand-side vector, numerically forcing uh(xi) ≈ DirVal(i) while keeping the matrix symmetric and its sparsity unchanged.

2.3 Architecture of the implemented code

The main script (main_fem.m) loops over the five meshes, running, for each one, the pipeline input_data → localBasis → stiffBuild → rhsBuild → imposeBC → system solution → error computation. The function input_data.m reads the connectivity (triang.dat), the coordinates (xy.dat) and the indices of the Dirichlet and Neumann nodes (dirnod.dat, neunod.dat) from disk, and defines both the exact solution ureal and the forcing term force as function handles, so that they can be evaluated pointwise at any point of the mesh without having to precompute vectors.

2.4 Solving the linear system: backslash and PCG

The penalized system is solved in two independent ways, so as to be able to cross-check their consistency: with MATLAB's backslash operator (direct factorization) and with the preconditioned conjugate gradient method (function pcgSolve.m), which uses as preconditioner the zero-fill incomplete Cholesky factorization IC(0) (ichol with the 'nofill' option). At each iteration pcgSolve.m records both the PCG “internal” residual log10(||rk||/||b||) and the true residual log10(||b − Axk||/||b||), making it possible to verify that the two quantities remain aligned even after many iterations (absence of numerical cancellation).

The following table reports, for each mesh, the norm difference between the direct solution uh and the one obtained with PCG, and the number of PCG iterations needed to reach the required tolerance ($tol = 10^{-8}$):

Mesh	‖uh − uh,PCG‖	PCG Iterations
mesh0	5.522e-09	8
mesh1	1.235e-08	15
mesh2	6.658e-09	29
mesh3	7.295e-08	55
mesh4	8.935e-08	109

The two solutions agree, in all cases, to about 8 significant digits: the discrepancy is of the order of the set tolerance rather than machine precision, which confirms that the stopping criterion is what determines the residual difference, not an assembly error. The number of PCG iterations grows in an almost linear fashion (roughly doubling at each refinement, 8→15→29→55→109): this behaviour is consistent with the fact that the condition number of the IC(0)-preconditioned stiffness matrix scales as $O(1/h)$ (versus $O(1/h^2)$ for the

unpreconditioned matrix), so the number of PCG iterations needed to reach a given tolerance grows roughly as 1/h, i.e. it doubles at every halving of h, in excellent agreement with what is observed.

2.5 Numerical results and convergence analysis

The L^2 norm error between the numerical and exact solutions $\varepsilon h = (\Sigma i\ (uh(i) - u(i))^2\ Ai)^{(1/2)}$ is reported, for each refinement level, in the following table together with the ratio between successive errors:

Mesh	h	L^2 Error (εh)	Ratio $\varepsilon h_{-1}/\varepsilon h$
mesh0	7.1113e-01	2.5124e-01	–
mesh1	3.5557e-01	5.1095e-02	4.9171
mesh2	1.7778e-01	1.1994e-02	4.2601
mesh3	8.8892e-02	2.9470e-03	4.0699
mesh4	4.4446e-02	7.3346e-04	4.0179

The theory for P1 elements (Nitsche-Aubin estimate) predicts $||u - uh||L^2 \leq C\ h^2\ |u|H^2$, from which the expected ratio between successive errors (h halved) tends to $2^2 = 4$. The observed values (4.92, 4.26, 4.07, 4.02) converge monotonically towards 4 as the refinement increases: the initial deviation (mesh0→mesh1 level) is physiological and is due to the fact that on the coarsest meshes the dominant error term is not yet asymptotically pure $O(h^2)$ (non-negligible higher-order contributions are present). The code therefore fully confirms the expected theoretical convergence order, as explicitly confirmed by the global convergence order found: 2.0956.

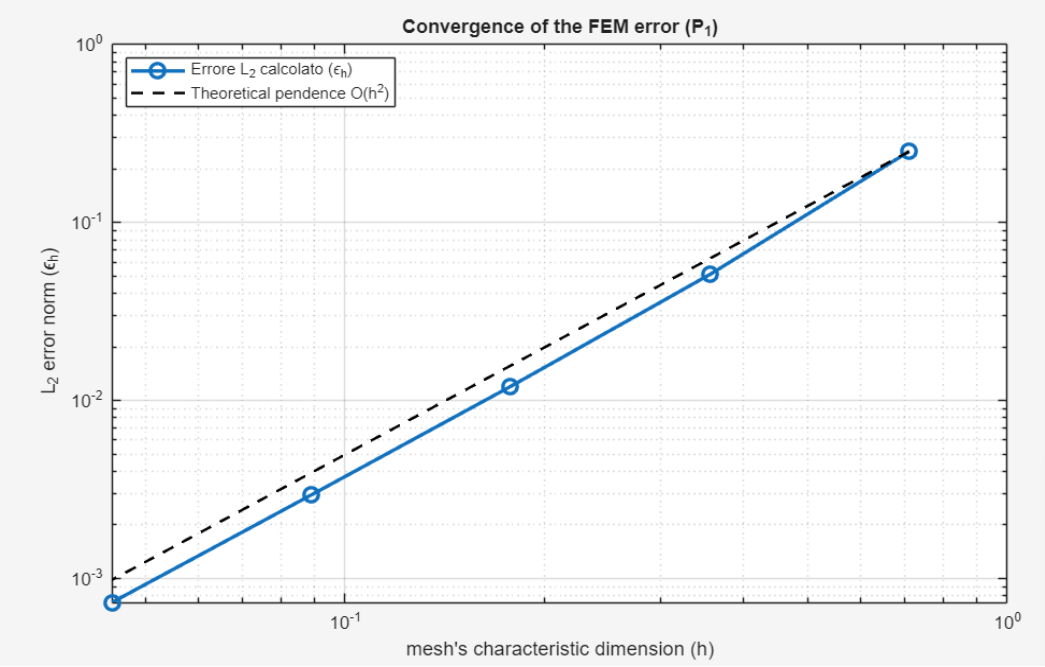


Figure 2.1 – Convergence of the FEM error in L^2 norm on a log-log scale, compared with the theoretical slope $O(h^2)$.

The following figures show the numerical solution uh, the exact solution u and the error function u − uh on the finest mesh mesh4: the two surfaces are visually indistinguishable, and the error takes values of the order of 10^{-3}, concentrated mainly near the corners of the domain where the solution is less regular.

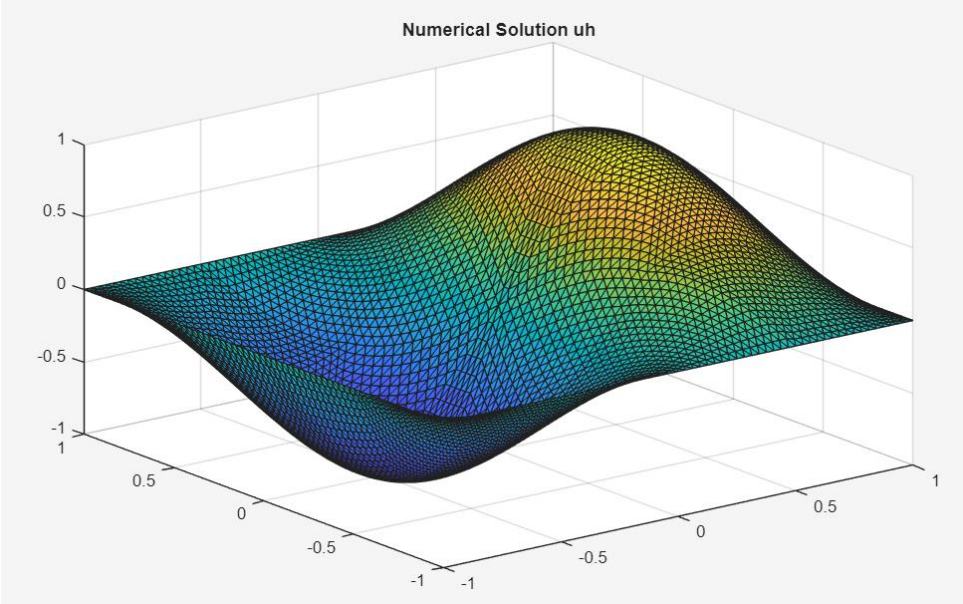


Figure 2.2 – Numerical solution uh on mesh4.

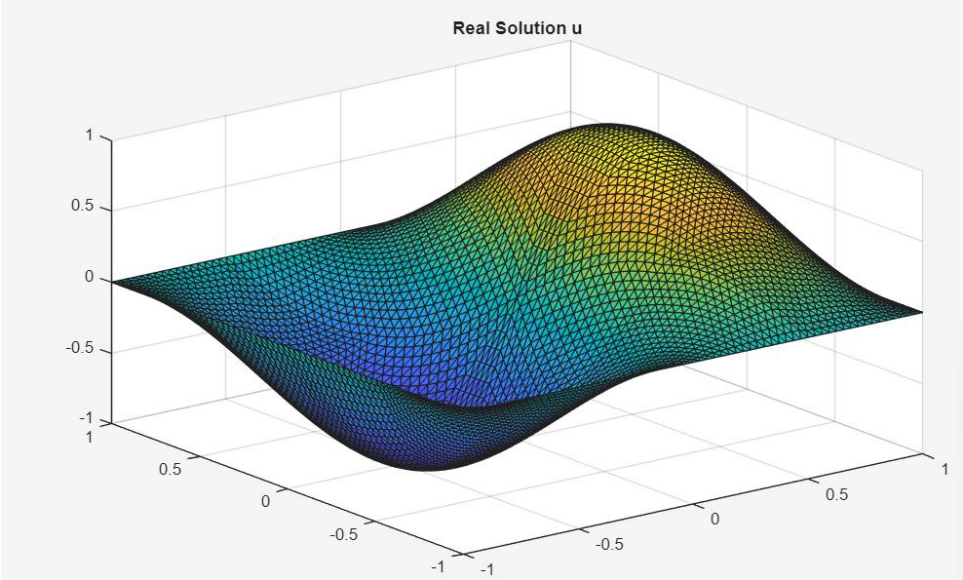


Figure 2.3 – Exact solution u on mesh4.

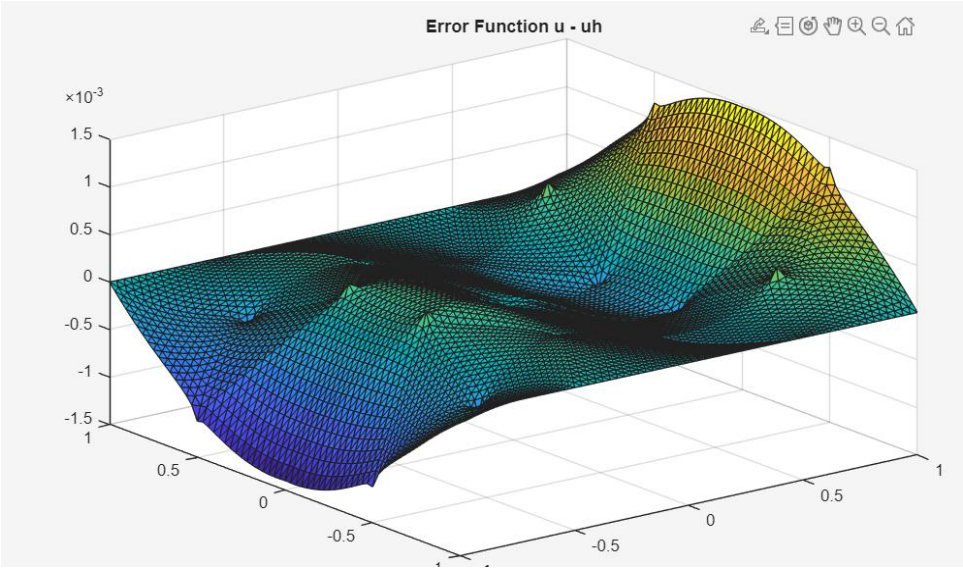


Figure 2.4 – Error function u − uh on mesh4 (note the order of magnitude 10^{-3}).

Finally, the convergence of the PCG method is shown on a semi-logarithmic scale for all five meshes: the decay of the relative residual is essentially linear (on a log scale) on each mesh, but with an increasingly gentler slope as the number of nodes grows, consistent with the increase in condition number already discussed. The comparison between the PCG "internal" residual and the true residual b − Axk, shown for mesh0 and mesh4, reveals an almost perfect overlap of the two curves throughout the iterations: this confirms that the IC(0) preconditioner does not introduce significant numerical cancellation, even after about a hundred iterations on mesh4.

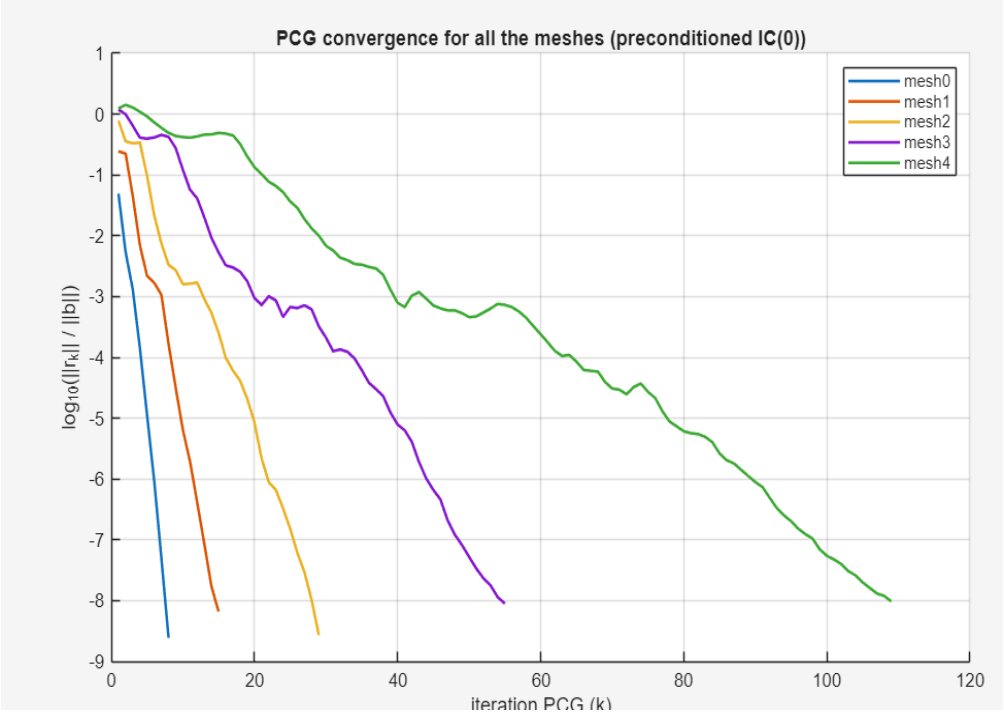


Figure 2.5 – PCG convergence (IC(0) preconditioner) for all meshes, semi-logarithmic scale.

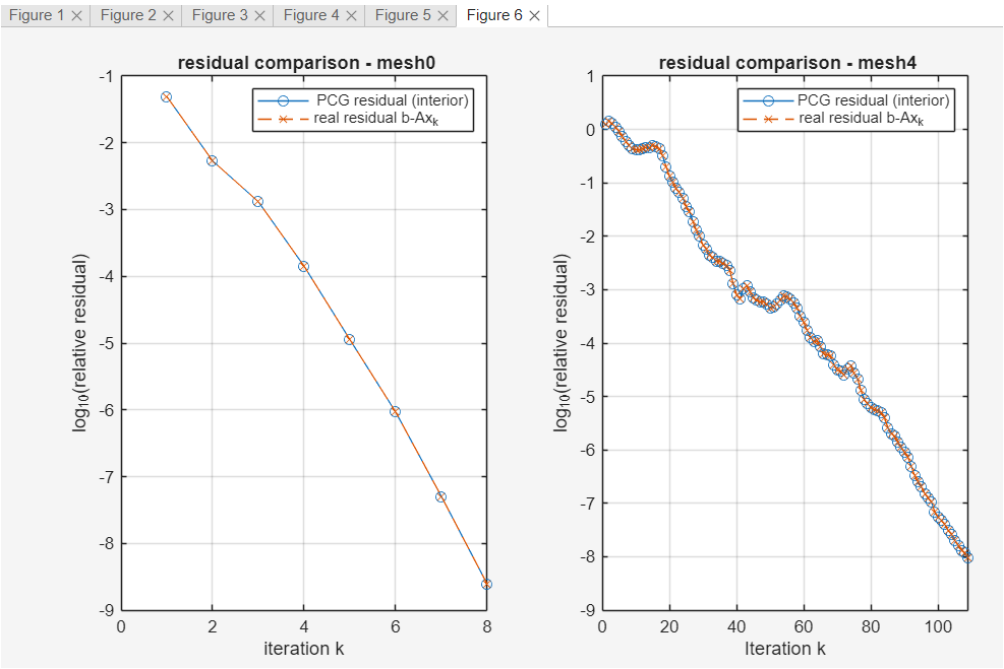


Figure 2.6 – Comparison between the internal PCG residual and the true residual b − Axk for mesh0 (left) and mesh4 (right).

3 Project 2 – Convection-Diffusion Equation

3.1 Problem formulation

The second project addresses the stationary, reaction-free version of the convection-diffusion-reaction equation, on the square domain Ω = (0,1)×(0,1):

$$-D\Delta u + \beta \cdot \nabla u = 0 \ \text{in } \Omega, \qquad \beta = (1,3)^T,$$

with purely geometric Dirichlet conditions, defined so as to introduce a boundary layer near the edge y = 1:

$$u = 1 \ \text{on } \Gamma 1 = \{x=0,\ 0 \le y \le 1\} \cup \{y=0,\ 0 \le x \le 0.3\}; \qquad u = 0 \ \text{on } \Gamma 2 = \partial\Omega \setminus \Gamma 1.$$

Since no closed-form analytical solution is available (the boundary data are discontinuous at the corners joining Γ1 and Γ2), the code cannot be verified by a direct comparison with an exact solution, but rather through a qualitative analysis of the numerical solution's behaviour as the diffusion coefficient D varies, and by checking that the maximum principle (0 ≤ u ≤ 1) is respected.

3.2 Galerkin formulation and transport matrix

Applying the weak formulation and substituting the Galerkin expansion uh = Σ ujϕj, the discrete problem leads to the linear system (A + B)u = 0, in which A is the usual stiffness matrix (identical, up to the coefficient D, to that of Project 1), while B is the transport matrix, with entries

$$b_{ij} = \int \Omega \ (\beta x \, \partial \phi_j / \partial x + \beta y \, \partial \phi_j / \partial y) \ \phi_i \, dx.$$

Unlike A, B is not symmetric: the system A+B therefore inherits the sparsity but not the symmetry of the stiffness matrix. The function transportBuild.m exploits the fact that, for P1 elements, the gradient of ϕj is constant on the triangle and the integral of ϕi over T always equals Area(T)/3 regardless of i: column j of the local matrix therefore has the same value repeated across the three rows, reducing the local computation to a single evaluation per column, val = (βx·Bloc(el,j) + βy·Cloc(el,j))/6.

3.3 Implementation notes

The function input_data2.m features two implementation choices worth noting. First, since the provided meshes are defined on the [−1,1]×[−1,1] domain of Project 1, while the Project 2 text explicitly requires Ω = [0,1]×[0,1], the node coordinates are rescaled with an affine transformation (translation + scaling) that maps the domain onto the unit square without altering the topology of the triangulation. Second, since the boundary conditions of this project are defined in a purely geometric way (rather than via the dirnod.dat/neunod.dat files), the function itself identifies the boundary nodes through a criterion on the coordinates (tolerance 10^{-8}) and classifies them into Γ1/Γ2 according to their position, explicitly handling the corner (0,1), the point of discontinuity of the boundary data, which is assigned by convention to Γ1.

The Dirichlet conditions are imposed using the same penalty method as in Project 1 (function imposeBC2.m), but with a more conservative penalty parameter, ε = 10^{-10} instead of 10^{-15}: as discussed in the project notes, a value of ε too small relative to machine precision causes, when the system is solved with iterative methods, a relative residual that artificially already appears below tolerance from the first iteration (due to numerical cancellation in the norm ||b||≈λ), even though the solution is not correct; ε = 10^{-10} represents the best compromise between the accuracy of the constraint imposition and the numerical stability of the solver. The system is nevertheless solved, in the script, with MATLAB's backslash operator.

3.4 Solution without stabilization: onset of oscillations

The code first solves the unstabilized problem for three decreasing values of the diffusion coefficient, D = 1, 10^{-1}, 10^{-3}, keeping the transport field $\beta = (1,3)^T$ fixed on mesh mesh2 ($h = 8.8892\times10^{-2}$).

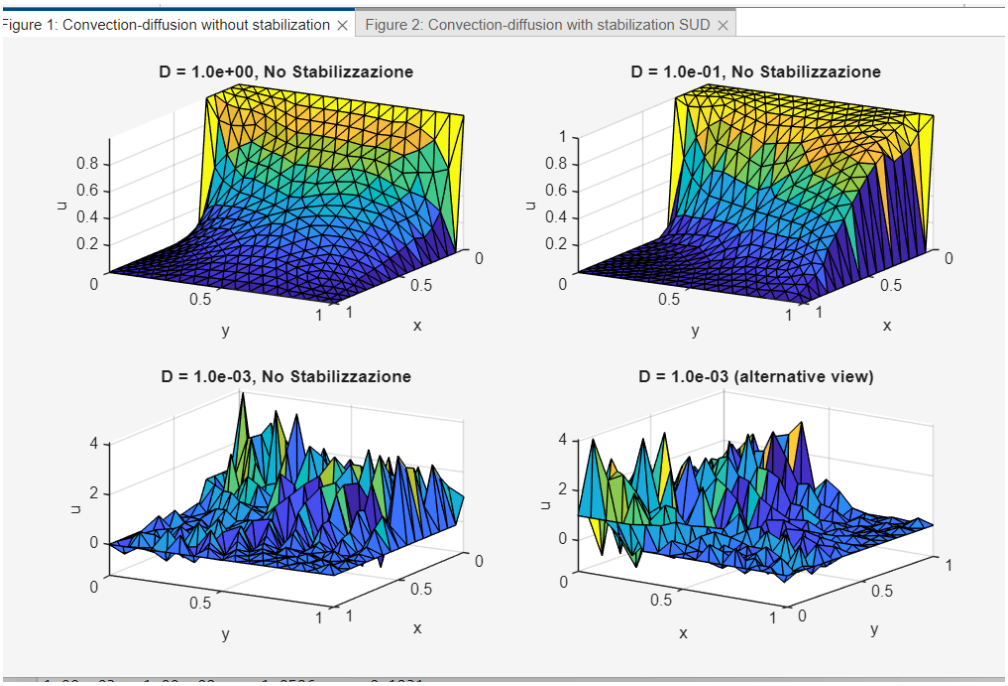


Figure 3.1 – Convection-diffusion solution without stabilization for D = 1, 10^{-1}, 10^{-3} (two views).

For D = 1 and D = 10^{-1} the solution is well-behaved and respects the maximum principle. For D = 10^{-3}, however, convection markedly dominates diffusion and the solution shows strong, unphysical spurious oscillations, with values that lie well outside the interval [0,1] (as quantified below). This behaviour is explained by Céa's error estimate for problems that are not uniformly coercive with respect to D, which involves the constant $\gamma/\alpha = (1+C\Omega^2)(1 + C\Omega|\beta|/D)$: when $|\beta| \gg D$ this constant — and with it the local Péclet number $Pe = C\Omega|\beta|/D$ — diverges, degrading the a priori error estimate even though the discrete solution exists and is unique (since the system remains invertible).

To quantitatively identify the threshold at which the oscillations appear, the code scans D on a logarithmic grid between 1 and 10^{-3}, checking at each step whether the solution violates the continuous maximum principle ($0 \le u \le 1$) beyond a tolerance of 10^{-2}. The critical value identified and the corresponding grid "pseudo-Péclet number" are:

Quantity	Value
Dcrit (threshold for the onset of oscillations)	7.0170×10^{-2}
h (mesh2)	8.8892×10^{-2}
$Peh = \|\beta\|\cdot h / (2\cdot Dcrit)$	2.0030

The experimentally identified value Peh ≈ 2 is in excellent agreement with the classical stability criterion for the standard Galerkin scheme applied to convection-diffusion problems, according to which oscillations appear for grid Péclet numbers above unity: the scan therefore numerically confirms, on the mesh used, the expected theoretical threshold.

The following table summarizes the maximum and minimum values of the solution obtained in the various cases, before and after stabilization (discussed in the next section): note how, without stabilization, the case

$D = 10^{-3}$ produces a marked overshoot/undershoot (max(u) = 4.11, min(u) = −1.25), far outside the physical interval [0,1].

D	δ (stabilization)	max(u)	min(u)
1.00e+00	0 (none)	1.0000	0.0000
1.00e-01	0 (none)	1.0000	0.0000
1.00e-03	0 (none)	4.1053	−1.2532
1.00e-03	5.00e-02	1.0226	−0.0953
1.00e-03	1.00e-02	1.0526	−0.1231
1.00e-03	1.00e-03	1.9670	−0.3312

3.5 Streamline-Upwind Diffusion (SUD) stabilization

To counter the oscillations, SUD stabilization is introduced, adding numerical diffusion to the system along the direction of the streamlines of the field β. The function stabBuild2.m assembles the stabilization matrix S, again exploiting the constancy of the gradient of ϕ on the triangle:

$$s_{ij}^{T} = \delta / (4 \cdot Area(T)) \cdot (\beta \cdot \nabla \phi_i)(T) \cdot (\beta \cdot \nabla \phi_j)(T),$$

with δ an empirical calibration coefficient (treated as a constant in the code, but also set up for an element-dependent "physical" variant, $\delta_T = \delta_0 \cdot h_T / (2|\beta|)$). The matrix S is symmetric and positive semi-definite and is added to the system as an additional diffusive contribution: (A + B + S)u = 0.

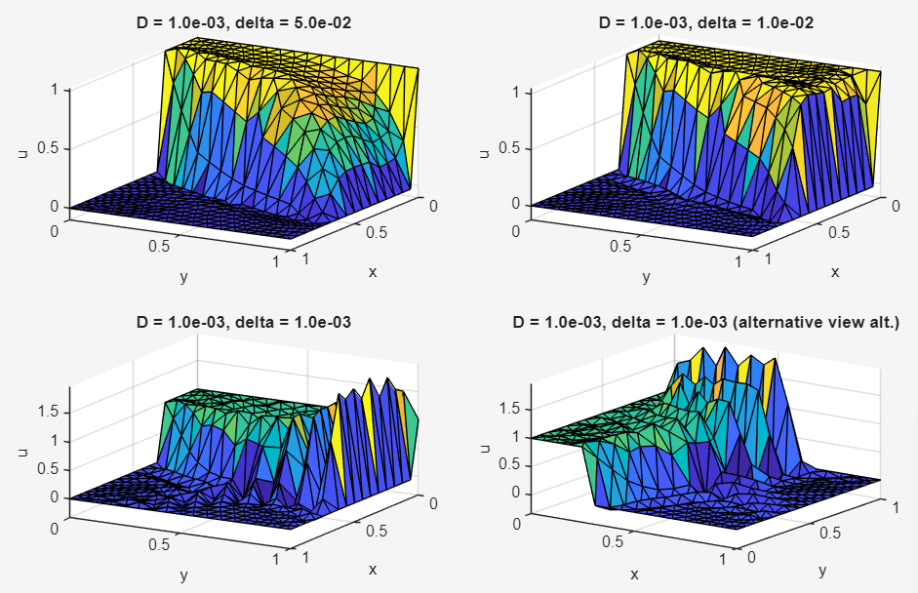


Figure 3.2 – Solution with SUD stabilization for $D = 10^{-3}$ and $\delta = 5\times10^{-2}, 10^{-2}, 10^{-3}$ (two views for $\delta=10^{-3}$).

The table and figure above clearly show the typical trade-off of SUD stabilization: for $\delta = 5\times10^{-2}$ the oscillations are almost completely suppressed (max(u) = 1.02, min(u) = −0.10, very close to respecting the maximum principle), but at the cost of more pronounced numerical diffusion, which smooths the boundary layer beyond what is physically expected; for $\delta = 10^{-3}$ the oscillations clearly persist (max(u) = 1.97, min(u) = −0.33), a sign that the added numerical diffusion is insufficient to compensate for the convective dominance. The intermediate value $\delta = 10^{-2}$ represents the best compromise between the two effects among those tested, significantly reducing the overshoot (max(u)=1.05, min(u)=−0.12) while still keeping a well-defined

boundary layer. This confirms what theory predicts: calibrating the stabilization parameter is an empirical trade-off between accuracy (little numerical diffusion) and stability (suppression of oscillations), and there is no universally optimal value of δ — it must be tuned case by case.

4 Project 3 – Parabolic Equation

4.1 Problem formulation

The third project deals with the parabolic heat equation in two spatial dimensions, with homogeneous Dirichlet conditions over the entire boundary:

$$\partial u/\partial t = \Delta u + f(x,y,t) \ \text{in } \Omega\times(0,T], \quad u = 0 \ \text{on } \partial\Omega, \quad \Omega = (-1,1)\times(-1,1).$$

The code uses the manufactured solution

$$u(x,y,t) = e^{-t}\sin(\pi x)\cos(\pi y/2), \quad f(x,y,t) = (5\pi^2/4 - 1)\, e^{-t}\sin(\pi x)\cos(\pi y/2),$$

with time horizon T = 1, which allows a pointwise comparison with the numerical solution at every instant.

4.2 FEM discretization and θ-method

The P1 Galerkin spatial discretization, applied to the semi-discrete weak formulation, leads to the ODE system M u' + A u = b(t), where A is the usual stiffness matrix and M the consistent mass matrix, with local contribution Mloc = (Area(T)/12)·[[2,1,1],[1,2,1],[1,1,2]], assembled in sparse form by massBuild3.m via triplets (I,J,V), analogously to stiffBuild3.m for the stiffness matrix.

Time integration uses the θ-method, which reduces to the explicit Euler scheme for θ = 0, to the implicit scheme for θ = 1, and to the Crank-Nicolson scheme for θ = 1/2:

$$(1/\Delta t \cdot P + \theta A)\, u^{n+1} = (1/\Delta t \cdot P - (1-\theta)A)\, u^n + \theta\, b^{n+1} + (1-\theta)\, b^n.$$

4.3 Implementation notes: solveParabolic.m

The project's central function, solveParabolic.m, takes the mesh, θ, the time step Δt, the horizon T and the handles of the functions uex/force, and returns the $L^2(0,T;L^2(\Omega))$ norm of the space-time error (computed by integrating in time, via trapz, the square of the spatial error weighted by the nodal areas nodeArea, obtained from nodeAreaAssemble.m), as well as the final solution uh(T) and the time mesh used. Some relevant implementation choices: the step Δt is internally redefined as T/round(T/Δt), so that it is always an exact divisor of T; the system matrix LHS = (1/Δt)P + θA, being constant in time (for a fixed Δt), is factorized only once with decomposition(LHS,'lu') and the factorization is reused at every time step, significantly reducing the computational cost compared to a repeated backslash solve at every iteration; the homogeneous Dirichlet conditions are imposed on the matrix with imposeBC3.m (which zeroes out the row and column of the constrained node and sets 1/ε on the diagonal, $\varepsilon = 10^{-15}$) and on the auxiliary unknown vector simply by setting rhs(DirNod)=0 at every step.

The function input_data3.m, unlike input_data.m in Project 1, merges the sets dirnod and neunod (or the set dirtot, if present) into a single vector of Dirichlet nodes, since in Project 3 the entire boundary ∂Ω is of homogeneous Dirichlet type.

4.4 Spatial convergence

With a very small time step fixed ($\Delta t = 10^{-4}$, so as to make the temporal error negligible) and varying the mesh from mesh0 to mesh4, the space-time error was computed for $\theta = 1$ (implicit Euler) and $\theta = 0.5$ (Crank-Nicolson):

Mesh	h	Error (θ=1)	Error (θ=0.5)
mesh0	5.0285e-01	1.0144e-01	1.0154e-01
mesh1	2.5142e-01	3.0472e-02	3.0522e-02
mesh2	1.2571e-01	7.9624e-03	7.9656e-03
mesh3	6.2856e-02	2.0009e-03	2.0036e-03
mesh4	3.1428e-02	4.9912e-04	5.0166e-04

The spatial convergence orders estimated by log-log regression are:

- $\theta = 1$ (implicit Euler): order 1.9527 (expected ≈ 2)
- $\theta = 0.5$ (Crank-Nicolson): order 1.9511 (expected ≈ 2)

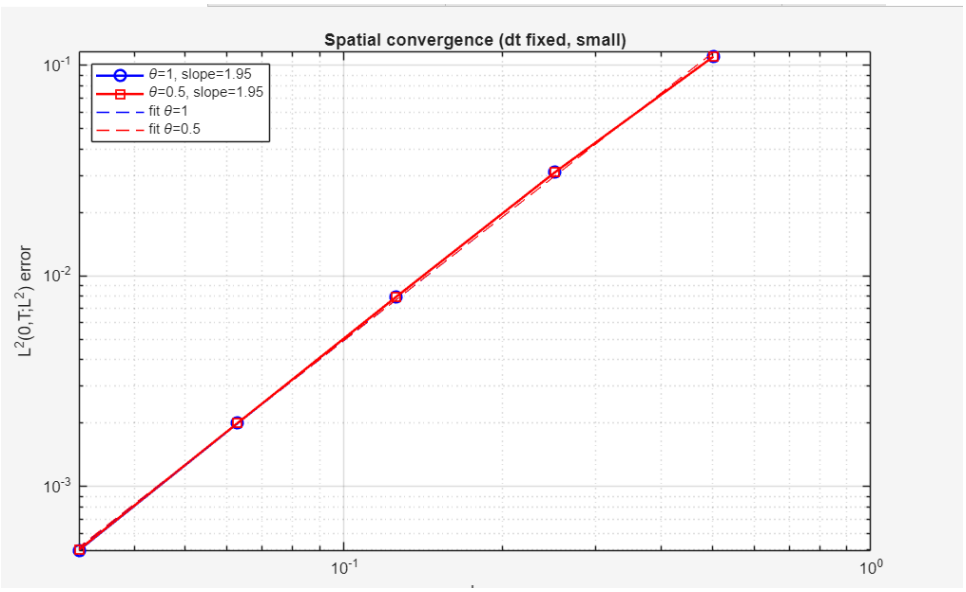


Figure 4.1 – Spatial convergence (fixed, small Δt) for ϑ = 1 and ϑ = 0.5, both with slope ≈ 2.

In both cases the spatial error is dominated by the P1 interpolation error, $O(h^2)$, as predicted by theory: the two curves practically overlap, since with such a small Δt the temporal error ($O(\Delta t)$ or $O(\Delta t^2)$ depending on θ) is completely negligible compared to the spatial one, and the plot correctly isolates only the h-behaviour.

4.5 Direct temporal convergence

Fixing the finest mesh (mesh4) and varying $\Delta t \in \{0.2, 0.1, 0.05, 0.025, 0.0125, 0.00625\}$, we obtain:

Δt	Error (θ=1)	Error (θ=0.5)
0.20000	4.1656e-03	7.1711e-04
0.10000	1.9292e-03	5.5394e-04
0.05000	7.4217e-04	5.1468e-04
0.02500	1.3305e-04	5.0493e-04
0.01250	1.8749e-04	5.0248e-04
0.00625	3.4352e-04	5.0187e-04

- θ = 1: estimated order 0.8734 (expected ≈ 1)
- θ = 0.5: estimated order 0.0864 (expected ≈ 2)

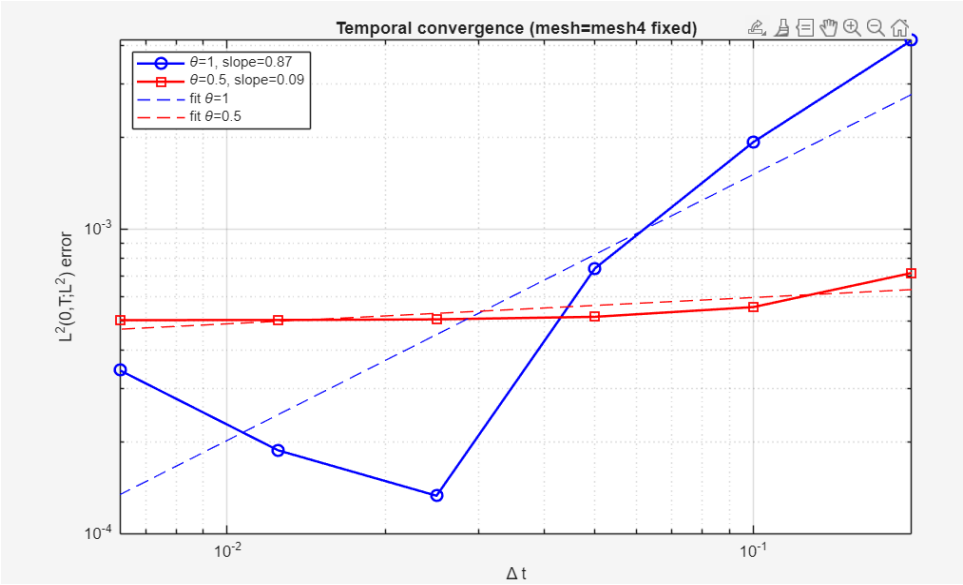


Figure 4.2 – "Direct" temporal convergence against the manufactured solution, fixed mesh4.

The result for θ = 1 is reasonably close to what is expected (0.87 versus 1), but the one for θ = 0.5 is clearly at odds with theory (0.09 versus an expected 2): looking at the table, one indeed notices that for θ = 0.5 the error remains essentially constant (≈ 5×10^{-4}) over the entire range of Δt explored, instead of decreasing quadratically. The explanation is that, since mesh4 is fixed at a finite value of h, the spatial error $O(h^2)$ (of the order of 5×10^{-4}, see the spatial convergence table) represents, for the Crank-Nicolson scheme — whose temporal error decays as $O(\Delta t^2)$ — an actual "floor" below which the total error can no longer drop, even by further reducing Δt: already at Δt = 0.2 the Crank-Nicolson temporal error is comparable to the spatial error, and for smaller Δt the total error saturates at the value of the spatial error, completely masking the true temporal convergence order. For θ = 1, whose temporal error $O(\Delta t)$ decreases more slowly, the spatial floor is reached only for the smallest Δt values tested, and indeed the table shows the error rising again (or at least no longer decreasing cleanly) for the last two values of Δt: this "bounce" contaminates the slope estimate computed over the whole range, explaining why even the order for θ = 1 turns out to be slightly lower than expected.

4.6 Self-convergence (Richardson) in time

To correctly isolate the temporal convergence order, eliminating contamination from the spatial error, the solution obtained with step Δt is compared — on the same mesh4 — with the one obtained with step Δt/2 (self-convergence/Richardson technique): since the spatial error is the same in both cases (same mesh), the difference $\|u_h^{\Delta t} - u_h^{(\Delta t/2)}\|$ isolates the pure temporal error, even when this is well below the spatial floor.

Δt	‖u^Δt−u^(Δt/2)‖ (θ=1)	‖u^Δt−u^(Δt/2)‖ (θ=0.5)
0.200000	1.8007e-03	8.9911e-05
0.100000	8.5619e-04	2.2077e-05
0.050000	4.1647e-04	6.7938e-06
0.025000	2.0533e-04	3.1794e-06
0.012500	1.0194e-04	1.3407e-06
0.006250	5.0790e-05	1.2033e-07
0.003125	2.5350e-05	1.9774e-08

- θ = 1: self-convergence order 1.0226 (expected ≈ 1)
- θ = 0.5: self-convergence order 1.9226 (expected ≈ 2)

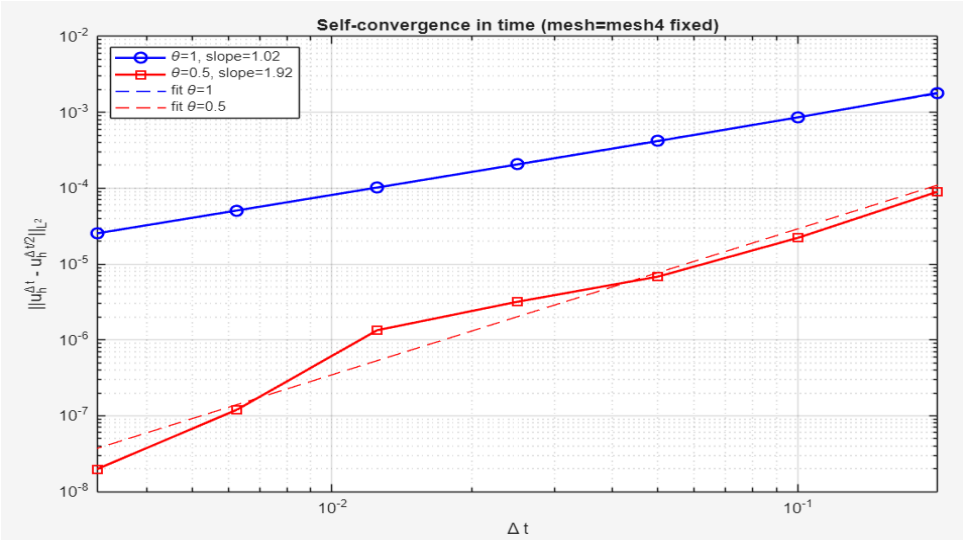


Figure 4.3 – Temporal self-convergence (comparison of u^Δt vs u^(Δt/2)), fixed mesh4.

The orders obtained with self-convergence are in excellent agreement with theory for both schemes, confirming the hypothesis put forward in the previous section: the disagreement observed in the "direct" temporal convergence (especially for Crank-Nicolson) was due solely to the floor imposed by the spatial error of the fixed mesh, and not to a flaw in the implementation of the temporal scheme. This comparison is a useful example of how the choice of verification metric (error against the exact solution vs. self-convergence)

can radically change the apparent outcome of a convergence test when the two errors (spatial and temporal) are not well separated in order of magnitude.

4.7 Stability of explicit Euler and the parabolic CFL limit

For θ = 0 (explicit Euler) the scheme is only conditionally stable, with threshold $\Delta t \leq 2/\lambda_{max}(P^{-1}A)$. For each mesh the code estimates λmax by solving the generalized eigenvalue problem Ax = λPx (function eigs), computes Δtstab = 0.9·(2/λmax) (safety factor 0.9) and, if the number of steps needed to reach T=1 does not exceed 20000, solves the problem with Δtused = T/⌈T/Δtstab⌉ = Δtstab:

Mesh	h	Δtstab	Δtused	Steps	Error
mesh0	5.0285e-01	1.364e-02	1.351e-02	74	1.1181e-01
mesh1	2.5142e-01	3.427e-03	3.425e-03	292	3.1230e-02
mesh2	1.2571e-01	8.402e-04	8.396e-04	1191	7.9923e-03
mesh3	6.2856e-02	2.076e-04	2.076e-04	4818	2.0091e-03
mesh4	3.1428e-02	5.104e-05	5.103e-05	19595	5.0296e-04

The convergence order estimated at the stability limit is 1.9551 (expected ≈ 2, excellent).

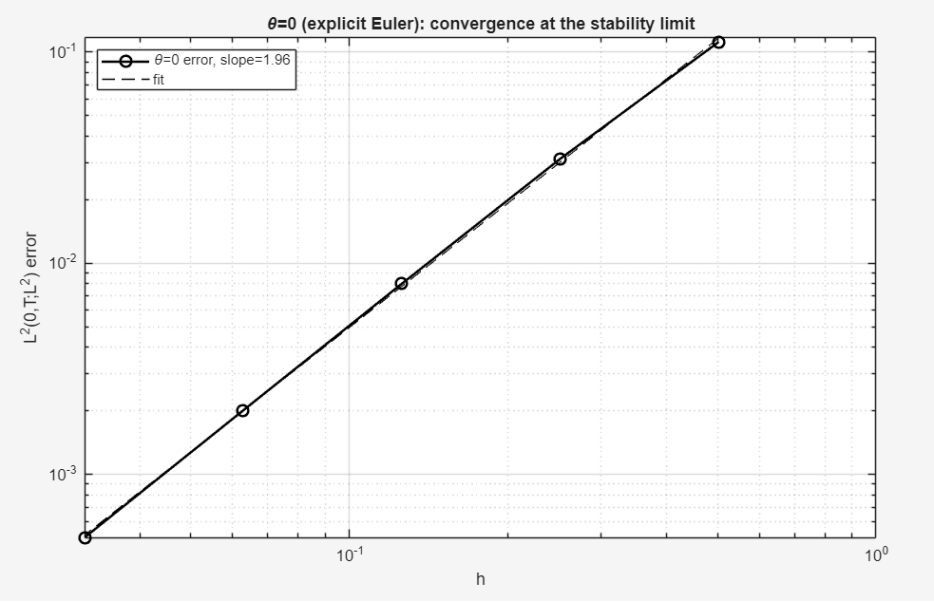


Figure 4.4 – Error convergence for ϑ=0 (explicit Euler) using, at each mesh, the step Δtstab allowed by stability.

The most significant finding of this section is the growth in the number of time steps required: going from mesh0 to mesh4 (h halved four times, hence reduced by a factor of 16), the step Δtstab shrinks by a factor of ≈267 (from 1.364×10^{-2} to 5.104×10^{-5}) and the number of steps grows accordingly from 74 to 19595. This is the expected $O(h^2)$ behaviour of the parabolic stability condition ($\Delta t \lesssim Ch^2/D$): halving h reduces the maximum allowed step by a factor of 4, and over four successive refinements the cumulative factor is of the

order of 4^4=256, in excellent agreement with the observed factor of ≈267. This concretely explains why, as noted in the project text, using explicit Euler on the finest meshes quickly becomes computationally prohibitive, motivating the preference for unconditionally stable schemes (implicit Euler or Crank-Nicolson) in practical applications.

5 Conclusions

The three projects made it possible to implement and validate, with progressively increasing complexity, the P1 Galerkin finite element method for elliptic and parabolic problems. In Project 1, the convergence of the L^2 norm error faithfully followed the theoretical order $O(h^2)$ (ratio between successive errors converging to 4), and the comparison between the direct solution and PCG preconditioned with IC(0) showed full consistency, with a number of iterations that grows as expected with the condition number. In Project 2, the appearance of numerical oscillations for convection-dominated problems was observed and quantified ($Pe_h \approx 2$ at the critical threshold), and the Streamline-Upwind Diffusion stabilization showed the expected trade-off between suppression of oscillations and the introduction of spurious numerical diffusion as the parameter δ increases. In Project 3, finally, the study of the spatial and temporal convergence of the θ-method highlighted an important methodological point: when the spatial error is not made negligible relative to the temporal one, the direct estimate of the temporal convergence order can be strongly distorted (as observed for Crank-Nicolson, apparent order 0.09 instead of 2), whereas the self-convergence (Richardson) technique, comparing solutions at Δt and $\Delta t/2$ on the same mesh, correctly isolates the true temporal order (1.02 for implicit Euler, 1.92 for Crank-Nicolson, both in excellent agreement with theory). Finally, the stability limit of explicit Euler confirmed the typical $O(h^2)$ scaling of the parabolic CFL condition, with a number of steps that grows by more than two orders of magnitude between the coarsest and finest meshes.